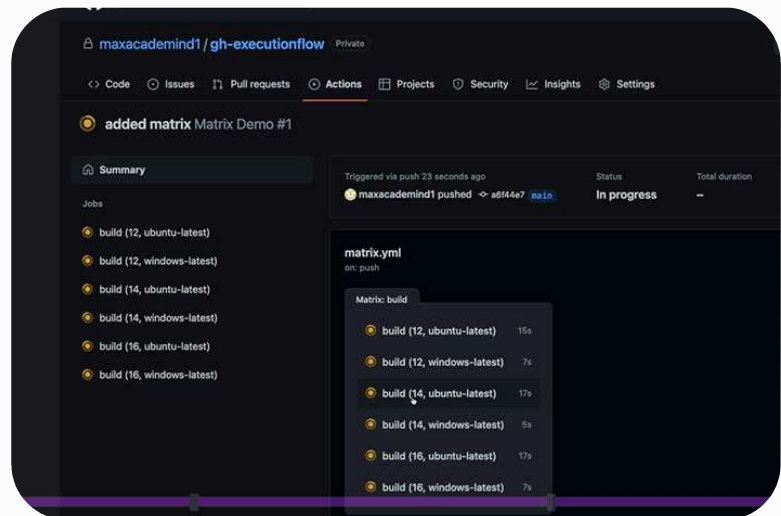


Today 11:57 AM



i need this code



The screenshot is showing a **GitHub Actions Matrix Strategy** workflow where the job runs for multiple combinations of **Node.js versions (12, 14, 16)** and **Operating Systems (Ubuntu and Windows)**.

Here's the equivalent `.github/workflows/matrix.yml` :

`</> YAML`

```
name: Matrix Demo

on:
  push:

jobs:
  build:
    runs-on: ${ matrix.operating-system }

    strategy:
      matrix:
        node-version: [12, 14, 16]
        operating-system:
          - ubuntu-latest
          - windows-latest

    steps:
      - name: Get Code
```

```

    uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: ${{ matrix.node-version }}

  - name: Display Build Information
    run: |
      echo "Operating System: ${{ matrix.operating-system }}"
      echo "Node Version: ${{ matrix.node-version }}"
      node --version

```

This matrix expands into 6 jobs automatically:

Node Version	Operating System
12	ubuntu-latest
12	windows-latest
14	ubuntu-latest
14	windows-latest
16	ubuntu-latest
16	windows-latest

Which matches the workflow shown in your screenshot:

```

build (12, ubuntu-latest)
build (12, windows-latest)
build (14, ubuntu-latest)
build (14, windows-latest)
build (16, ubuntu-latest)
build (16, windows-latest)

```



If you're following **Maximilian Schwarzmüller's GitHub Actions course**, this is the exact matrix workflow demonstrated in the "Matrix Strategy" section.



Today 1:24 PM

pls explain

Let's go through the workflow **line by line** so you understand exactly how the **Matrix Strategy** works in GitHub Actions.

</> YAML



```
name: Matrix Demo
```

- This is the **workflow name**.
- It appears in the **Actions** tab of your GitHub repository.

</> YAML



```
on:  
  push:
```

This tells GitHub:

Run this workflow every time someone pushes code to the repository.

For example:

```
git add .  
git commit -m "Added feature"  
git push
```



GitHub automatically starts this workflow.

jobs

</> YAML



```
jobs:
```

A workflow can contain one or more jobs.

Example:

Workflow



```
|  
├─ Build Job  
├─ Test Job  
└─ Deploy Job
```

In this example we only have one job.

</> YAML



```
build:
```

The job is named **build**.

runs-on

</> YAML



```
runs-on: ${ matrix.operating-system }
```

Normally we write

</> YAML



```
runs-on: ubuntu-latest
```

which means

Run only on Ubuntu.



But here we use

</> YAML



```
${ matrix.operating-system }
```

This means

"Take the operating system from the matrix."

GitHub will replace this with

```
ubuntu-latest
```



or

```
windows-latest
```



depending on the current matrix combination.

strategy

```
</> YAML
```



```
strategy:
```

A strategy tells GitHub

"How should this job be executed?"

One strategy is **matrix**.

```
</> YAML
```



```
matrix:
```

The matrix creates **multiple copies** of the same job.

Instead of writing six jobs manually, GitHub creates them automatically.

node-version

```
</> YAML
```



```
node-version: [12, 14, 16]
```

This means

```
Run using Node 12
```

```
Run using Node 14
```

```
Run using Node 16
```



operating-system

</> YAML



```
operating-system:
  - ubuntu-latest
  - windows-latest
```

This means

Run on Ubuntu
Run on Windows



Matrix Expansion

GitHub combines **every value** with every other value.

Think of it like multiplication.

Node Versions



12
14
16

×

Operating Systems



Ubuntu
Windows

↓

Result

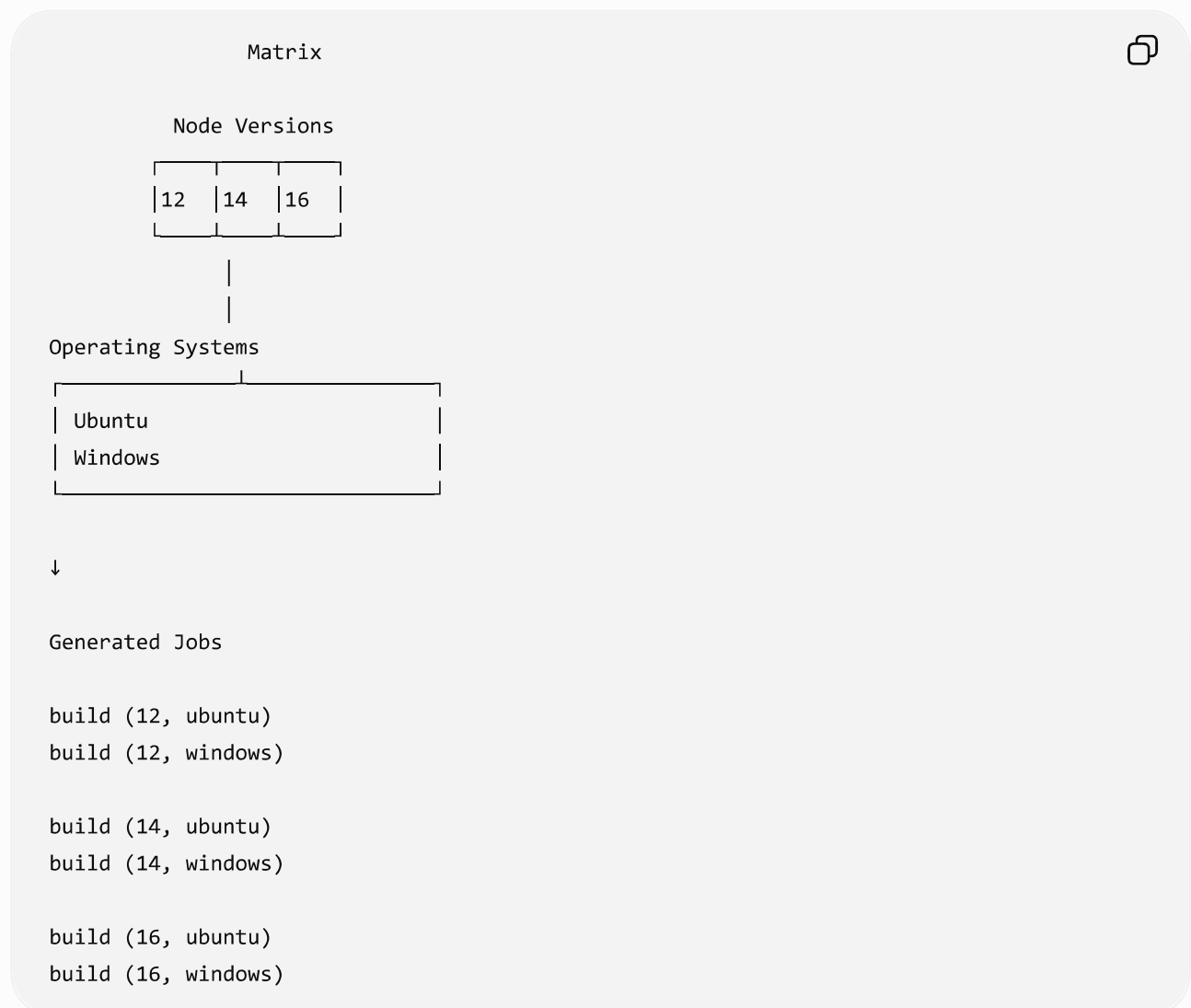
Node	OS
12	Ubuntu
12	Windows

Node	OS
14	Ubuntu
14	Windows
16	Ubuntu
16	Windows

Exactly **6 jobs**.

This is what you saw in the screenshot.

Visualization



GitHub starts all of these jobs **in parallel** (subject to runner availability).

Steps

Each generated job executes the same steps.

Step 1

</> YAML



```
- name: Get Code
  uses: actions/checkout@v4
```

This downloads your repository onto the GitHub runner.

Before this step:

Runner
(empty)



After this step:

Runner



```
project/
  package.json
  src/
  README.md
```

Without `checkout`, your workflow cannot access your project files.

Step 2

</> YAML



```
- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: ${{ matrix.node-version }}
```

This installs the Node.js version specified by the current matrix job.

For example:

Job 1:

Node 12
Ubuntu



GitHub installs

Node v12



Job 2:

Node 14
Windows



GitHub installs

Node v14



Job 3:

Node 16
Ubuntu



GitHub installs

Node v16



The expression

</> YAML



```
${{ matrix.node-version }}
```

is replaced automatically with the current value.

Step 3

</> YAML



```
- name: Display Build Information
  run: |
    echo "Operating System: ${{ matrix.operating-system }}"
```

```
echo "Node Version: ${ matrix.node-version }"
node --version
```

The `run` keyword executes shell commands.

Example output for one job:

```
Operating System: ubuntu-latest
Node Version: 14
v14.21.3
```



For another job:

```
Operating System: windows-latest
Node Version: 16
v16.20.2
```



What GitHub Actually Does Internally

Although you wrote **one job**, GitHub internally expands it into something like this:

YAML



```
build-1:
  runs-on: ubuntu-latest
  node-version: 12

build-2:
  runs-on: windows-latest
  node-version: 12

build-3:
  runs-on: ubuntu-latest
  node-version: 14

build-4:
  runs-on: windows-latest
  node-version: 14

build-5:
  runs-on: ubuntu-latest
  node-version: 16

build-6:
```

```
runs-on: windows-latest
node-version: 16
```

Instead of writing six nearly identical jobs yourself, the matrix strategy generates them automatically.

Why Use Matrix Strategy?

Suppose your application should work on multiple platforms and Node.js versions. Without a matrix, you'd have to duplicate the same job many times. With a matrix, you define the variables once, and GitHub creates every combination for you.

Benefits include:

- **Less repetition:** one job definition instead of many.
- **Broader testing:** verify compatibility across environments.
- **Parallel execution:** combinations can run simultaneously, reducing overall CI time.

This is why matrix builds are commonly used for testing libraries or applications that support multiple operating systems, language versions, or configurations.

